

Discord Clone

Backend Integration Guide

For Frontend Team (React / Next.js)

Architecture:	Java Spring Boot (Microservices)
API Gateway:	Spring Cloud Gateway (Port 8000)
Authentication:	Keycloak (OAuth2 / JWT)
Real-time:	STOMP over WebSockets & RabbitMQ
Storage:	PostgreSQL & MinIO
State:	Redis (Presence & Voice Lists)

Generated on: July 2024

Contents

1	Base URL & Authentication	2
1.1	1.1. Register	2
1.2	1.2. Login	2
1.3	1.3. Refresh Token	2
1.4	1.4. Logout	3
2	Media Handling (File Uploads)	3
2.1	2.1. Upload File	3
2.2	2.2. Accessing Files	3
3	Real-Time Communication (WebSocket)	4
3.1	3.1. Connection Setup	4
3.2	3.2. Subscriptions (Complete List of Events)	4
3.3	3.3. Publishing (Sending a Message)	4
3.4	3.4. Read Receipts (Blue Ticks)	5
4	Servers (Groups) & Channels	5
4.1	4.1. Servers	5
4.2	4.2. Server Members	5
4.3	4.3. Channels	6
5	Topics (Threads)	6
6	Roles & Permissions (Dynamic RBAC System)	6
6.1	6.1. Changing a User's Role	7
7	Messages (REST Endpoints)	7
7.1	7.1. Load History	7
7.2	7.2. Search (Updated Behavior)	7
7.3	7.3. Scheduled Messages (New Feature)	7
7.4	7.4. Edit & Delete	8
8	Direct Messages (DMs)	8
9	User Profiles	9
10	Presence & Online Status (Redis Module)	9
11	Voice Chat / WebRTC Signaling	9
11.1	11.1. Voice Flow	10
11.2	11.2. API & Payload	10
11.3	11.3. Getting Voice Members	10
12	Data Models (TypeScript Interfaces)	11

1 Base URL & Authentication

All HTTP requests (except Register and Login) and WebSocket connections require authentication.

- **Base URL:** `http://localhost:8000`
- **Header:** `Authorization: Bearer <accessToken>`
- **Important Note (Token Refresh):** Access tokens are short-lived. If any endpoint returns 401 Unauthorized, **immediately** call the Refresh Token endpoint. If the refresh also returns 401, redirect the user to the Login page.

1.1 1.1. Register

```
1 POST /api/v1/users/register
```

```
1 // Request Body
2 {
3   "username": "sepehr",
4   "email": "sepehr@example.com",
5   "password": "StrongPass123!",
6   "firstName": "Sepehr",
7   "lastName": "Yazdi"
8 }
9 // Response: 201 Created
10 "User registered successfully"
```

1.2 1.2. Login

```
1 POST /api/v1/users/login
```

```
1 // Request Body
2 { "username": "sepehr", "password": "StrongPass123!" }
3
4 // Response: 200 OK
5 {
6   "accessToken": "eyJhbGciOi...",
7   "refreshToken": "eyJhbGciOi...",
8   "expiresIn": 300
9 }
```

Frontend Action: Store `accessToken` in memory/state management (Redux/Zustand). Securely store `refreshToken` to use for the endpoint below.

1.3 1.3. Refresh Token

```
1 POST /api/v1/users/refresh
```

```

1 // Request Body
2 { "refreshToken": "eyJhbGciOi..." }
3
4 // Response: 200 OK (Exactly like Login Response)
5 {
6   "accessToken": "new_access_token...",
7   "refreshToken": "new_refresh_token...", // Keycloak invalidates
8   "expiresIn": 300
9 }

```

1.4 1.4. Logout

```
1 POST /api/v1/users/logout
```

```

1 // Request Body
2 { "refreshToken": "eyJhbGciOi..." }
3 // Response: 204 No Content

```

Note: Invalidates the session server-side. The user will be logged out across all open tabs.

2 Media Handling (File Uploads)

Golden Rule: Never send file binaries (Multipart) directly to the Chat or Voice services.

Workflow:

1. Upload the file to the Media route.
2. Receive a generated unique string filename.
3. Pass that string as `attachmentFileName` in your JSON payload to Chat/Auth APIs.

2.1 2.1. Upload File

```
1 POST /api/media/upload
```

- **Content-Type:** multipart/form-data
- **Form-Data Key:** file
- **Response:** 200 OK (String)

```
1 "1698765432_profile-pic.jpg"
```

2.2 2.2. Accessing Files

To display an image/file, use the GET endpoint: `GET /api/media/{fileName}`

This returns the raw binary stream. You can use this directly in `` or as an `<a href="..." />` for downloads.

3 Real-Time Communication (WebSocket)

We use STOMP over WebSockets. The `Authorization` header must be sent during connection.

3.1 3.1. Connection Setup

Connect your STOMP client to: `ws://localhost:8000/ws`

```
1 // Example Frontend Connection Code
2 const socket = new SockJS('http://localhost:8000/ws');
3 const stompClient = Stomp.over(socket);
4 stompClient.connect({ 'Authorization': 'Bearer ' + accessToken },
    onConnected);
```

3.2 3.2. Subscriptions (Complete List of Events)

Upon successful connection, you **must** subscribe to these paths based on your UI context:

- `/topic/channel/{channelId}`: Receive chat messages (Private, Group, Channel).
- `/user/queue/notifications`: Receive notifications (New DM message, Added to server).
- `/topic/server/{serverId}/events`: System events (Real-time Channel/Topic creation/deletion).
- `/topic/presence/global`: User status changes (Online/Offline).
- `/topic/voice/{channelId}`: Users joining/leaving a specific voice channel.
- `/user/queue/voice-signaling`: Receive WebRTC signaling payloads (Offer/Answer/ICE).
- `/topic/channel/{channelId}/read`: Receive read receipts (blue ticks).

3.3 3.3. Publishing (Sending a Message)

Publish to: `/app/chat/channel/{channelId}`

```
1 // Payload (LiveMessagePayload)
2 {
3   "content": "Hello guys!",
4   "attachmentFileName": "1698765432_image.png", // null if text-
      only
5   "topicId": 15 // null if not inside a topic
6 }
```

Critical Technical Note: The backend intercepts this, saves it to the PostgreSQL database, and **then** broadcasts it. Therefore, the payload you receive in your subscription will contain the real database id and the exact server-side `createdAt` timestamp.

3.4 3.4. Read Receipts (Blue Ticks)

When a user opens a chat and sees the messages, send this (no body required):

Publish to: `/app/chat/read/{channelId}`

Listeners on `/topic/channel/{channelId}/read` will receive a payload containing `channelId` and `readBy` username.

4 Servers (Groups) & Channels

4.1 4.1. Servers

```
1 POST /api/v1/servers
```

```
1 // Request Body
2 { "name": "Backend Team", "iconUrl": "1698765432_icon.png", "
  initialMemberUsernames": ["ali", "reza"] }
```

- GET `/api/v1/servers` (Get all servers for the logged-in user)
- GET `/api/v1/servers/{id}` (Get server details - includes channels, members, and roles)
- PUT `/api/v1/servers/{id}/name` (Update server name - Any member can do this)

```
1 { "name": "New Name" }
```
- DELETE `/api/v1/servers/{id}` (Delete server - Any member can delete. Cascades to all messages/files).

4.2 4.2. Server Members

```
1 POST /api/v1/servers/{serverId}/members
```

```
1 { "username": "newuser" }
```

Error Handling: If the target user has restricted group adds in their privacy settings, the backend returns 400 Bad Request: "This user has restricted others from adding them to servers/groups.". You must show this exact message to the user via a Toast/Alert.

Leave Server:

```
1 DELETE /api/v1/servers/{serverId}/leave
```

Rules: Any member can leave the server at any time to remove themselves, **EXCEPT** the server OWNER. If the OWNER attempts to leave, the backend returns a 400 Bad Request. Upon a successful 200 OK, remove the server from the user's local state immediately.

Ban Member (Kick/Remove Permanently):

```
1 POST /api/v1/servers/{serverId}/members/{targetUsername}/ban
```

Rules & Hierarchy Logic:

- Requires the `BAN_MEMBERS` permission in the requester's role.
- You cannot ban yourself.
- **No one** (not even other Admins) can ban the server OWNER.
- An Admin cannot ban another member who also possesses the `BAN_MEMBERS` permission (e.g., another Admin). **Only the OWNER** can bypass this and ban other Admins/Moderators.

4.3 Channels

```
1 POST /api/v1/channels
1 // Request Body (Requires 'MANAGE_CHANNELS' permission in the new
  RBAC system)
2 { "name": "general", "type": "TEXT", "serverId": 1 } // type: "
  TEXT" or "VOICE"
```

- `PUT /api/v1/channels/{channelId}/name` (Update name)
- `DELETE /api/v1/channels/{channelId}` (Delete channel and its messages)
- `PUT /api/v1/channels/{channelId}/media-restriction?isRestricted={true/false}` (Toggle media restriction for normal members)

5 Topics (Threads)

Topics allow sub-discussions inside a channel.

- `POST /api/v1/topics` (Requires `MANAGE_TOPICS` permission)

```
1 { "channelId": 1, "name": "Design Discussions" }
```

- `DELETE /api/v1/topics/{topicId}` (Deletes topic and automatically purges attached media files from MinIO).

6 Roles & Permissions (Dynamic RBAC System)

The role system is **no longer a static Enum**. Servers have custom, dynamically created roles containing a JSON array of permissions.

Standard Default Permissions Available: `SEND_TEXT`, `ATTACH_FILES`, `DELETE_OWN_MESSAGE`, `DELETE_ANY_MESSAGE`, `MANAGE_TOPICS`, `MANAGE_CHANNELS`, `MENTION_EVERYONE`, `MANAGE_ROLES`, `BAN_MEMBERS`. (Note: The OWNER role automatically gets a wildcard "*" permission which overrides all checks).

6.1 6.1. Changing a User's Role

```
1 PUT /api/v1/channels/{channelId}/members/role
```

```
1 // Request Body
2 {
3   "targetUsername": "ali",
4   "newRoleId": 5 // Send the Long ID of the role, NOT the name
5 }
```

Rules:

- Only a user with the `MANAGE_ROLES` permission can perform this.
- No one can change the `OWNER`'s role.

7 Messages (REST Endpoints)

Note: Real-time message sending is done **only** via WebSocket (Section 3.3). These endpoints are for loading history, searching, and management.

7.1 7.1. Load History

```
1 GET /api/v1/messages/channel/{channelId}
```

Returns messages in ascending order (oldest to newest).

7.2 7.2. Search (Updated Behavior)

```
1 GET /api/v1/messages/channel/{channelId}/search?keyword=hello&
    limit=20&cursor=xyz
```

Important Change: The search engine now looks **simultaneously** in both the content (text/caption) AND the `attachmentFileName` (filename).

```
1 // Response
2 {
3   "messages": [ ... ],
4   "nextCursor": "eyJtcyI6MTY5ODc2..." // Pass this in the next
5   request for page 2
6 }
```

7.3 7.3. Scheduled Messages (New Feature)

Send a message for the future.

- **Create:** `POST /api/v1/messages/schedule`


```

1      {
2          "channelId": 12,
3          "content": "Happy birthday!",
4          "scheduledAt": 1699000000000, // Unix Timestamp in the
           future
5          "attachmentFileName": null,
6          "topicId": null
7      }
8      // Response: Message object with status "SCHEDULED"

```

- **Edit:** PUT /api/v1/messages/schedule/{messageId}

```

1      { "content": "New text", "scheduledAt": 1699000050000 }

```

- **Cancel:** DELETE /api/v1/messages/schedule/{messageId}
(Status changes to CANCELED, it will NOT be sent).

7.4 7.4. Edit & Delete

- **Edit:** PUT /api/v1/messages/{messageId} (Sender only)

```

1      { "content": "Edited text" }
2      // Backend returns message with "isEdited": true

```

UI Action: If isEdited === true, render a small "(Edited)" tag next to the message.

- **Delete:** DELETE /api/v1/messages/{messageId}
(Sender can delete own with DELETE_OWN_MESSAGE perm, Admins can delete any with DELETE_ANY_MESSAGE perm).

8 Direct Messages (DMs)

DMs are treated as hidden channels under the hood.

```

1 POST /api/v1/dm/start

```

```

1 { "targetUsername": "ali" }
2
3 // Response: The Channel object representing the DM room
4 {
5     "id": 55,
6     "name": "DM",
7     "type": "DIRECT",
8     "server": null
9 }

```

UI Integration: Take the returned id (e.g., 55), and subscribe to the WebSocket topic /topic/channel/55. From now on, DMs work exactly like group channel messages.

9 User Profiles

- GET /api/v1/users/me (Get current user profile)
- GET /api/v1/users/{username} (View another user's public profile - no email included)
- PUT /api/v1/users/me (Update profile - username cannot be changed)

```
1  {
2    "firstName": "Sepehr",
3    "lastName": "Yazdi",
4    "bio": "Backend Developer",
5    "avatarUrls": ["url1.jpg", "url2.jpg"] // Send the full
6    array. Backend auto-deletes old images from MinIO
  }
```

- POST /api/v1/users/avatar?newAvatarUrl=xyz.jpg (Appends a single new image to the array without replacing others).

10 Presence & Online Status (Redis Module)

The backend uses Redis with TTL (Time-To-Live) for online status.

- **Heartbeat (Crucial):** The frontend **must** hit this endpoint every 30 seconds to reset the Redis timer. Otherwise, the user will automatically appear offline.

```
1 POST /api/v1/presence/online
```

- **Explicit Offline:** When closing the tab or logging out:

```
1 POST /api/v1/presence/offline
```

- **Listening to Events:** Subscribe to /topic/presence/global in WebSocket.

```
1 // Example Received Payload
2 { "eventType": "USER_ONLINE", "username": "ali", "
   voiceChannelId": null }
3 { "eventType": "USER_OFFLINE", "username": "reza", "
   voiceChannelId": null }
```

11 Voice Chat / WebRTC Signaling

The backend **DOES NOT** handle actual audio streams. Audio is strictly Peer-to-Peer (P2P) via WebRTC. The backend only routes the negotiation signaling messages.

11.1 11.1. Voice Flow

1. User A clicks a Voice Channel: POST /api/v1/presence/voice/join/{channelId} (Updates online list).
2. User A prepares their browser and generates an SDP Offer.
3. User A sends it to the backend: POST /api/v1/voice/signal
4. User B receives it via /user/queue/voice-signaling.
5. User B generates an SDP Answer and sends it to the same backend endpoint.
6. User A receives the answer and the P2P audio connection is established.

11.2 11.2. API & Payload

```
1 POST /api/v1/voice/signal
```

```
1 // For sending Offer or Answer
2 {
3   "type": "OFFER", // or "ANSWER"
4   "toUser": "ali",
5   "channelId": 15,
6   "sdp": "v=0\r\no=- 4611731400430051336 2 IN IP4..."
7 }
8
9 // For sending ICE Candidates (during negotiation)
10 {
11   "type": "ICE_CANDIDATE",
12   "toUser": "ali",
13   "channelId": 15,
14   "candidate": "candidate:842163049 1 udp 1677729535 192.168.1.5
15     43521 typ srflx..."
16 }
```

Note: Do **not** populate the `fromUser` field in the frontend; the backend securely extracts it from the JWT token to prevent identity spoofing.

11.3 11.3. Getting Voice Members

To show user avatars at the top of a voice channel:

```
1 GET /api/v1/presence/voice/members/{channelId}
```

```
1 // Response: Set<String>
2 ["sepehr", "ali", "reza"]
```

12 Data Models (TypeScript Interfaces)

Copy these interfaces directly into your frontend project for complete type-safety. They accurately reflect the final backend DTOs.

```
1 // --- Messages & Chat ---
2 export interface LiveMessagePayload {
3   id?: number;           // Populated after DB save by backend
4   content: string;
5   senderUsername: string;
6   createdAt?: number;    // Unix Timestamp (populated after DB
7                           save)
8   channelId: number;
9   topicId?: number | null; // Added for Thread support
10  attachmentFileName?: string | null;
11 }
12
13 export interface Message extends LiveMessagePayload {
14   isEdited: boolean;
15   status?: 'SENT' | 'SCHEDULED' | 'CANCELED'; // Added for
16   Scheduled Messages
17   scheduledAt?: number | null; // Added for Scheduled Messages
18   channel: Channel;
19   topic?: Topic | null;
20 }
21
22 // --- Servers & Channels ---
23 export interface Role { // Added (Dynamic RBAC System)
24   id: number;
25   name: string; // "OWNER", "ADMIN", "MEMBER", or custom names
26   permissionsJson: string; // JSON string array. Frontend doesn't
27                           need to parse, just display for admins
28 }
29
30 export interface ServerMember {
31   id: number;
32   username: string;
33   role: Role; // Changed! No longer an Enum, it is a Role Object
34 }
35
36 export interface Server {
37   id: number;
38   name: string;
39   iconUrl: string | null;
40   ownerUsername: string;
41   channels: Channel[];
42   members: ServerMember[];
43   roles?: Role[]; // List of available roles in this server (for
44                   Admin Panel)
45 }
46
47 export interface Channel {
```

```

44     id: number;
45     name: string;
46     type: "TEXT" | "VOICE" | "DIRECT";
47     server: Server | null;
48     mediaRestricted: boolean;
49     topics: Topic[];
50 }
51
52 export interface Topic {
53     id: number;
54     name: string;
55     channel: Channel;
56 }
57
58 // --- Users ---
59 export interface UserProfile {
60     id: string;
61     username: string;
62     email: string;
63     firstName: string;
64     lastName: string;
65     bio: string;
66     avatarUrls: string[];
67 }
68
69 // --- Events & Notifications ---
70 export interface ServerEventPayload {
71     eventType: "CHANNEL_CREATED" | "CHANNEL_DELETED" | "
        TOPIC_CREATED" | "TOPIC_DELETED";
72     serverId: number;
73     data: any; // Can be Channel object, Topic object, or just a
        numeric ID for deletions
74 }
75
76 export interface NotificationPayload {
77     type: "NEW_MESSAGE" | "ADDED_TO_SERVER";
78     senderUsername?: string;
79     targetChannelId?: number;
80     snippet?: string;
81     chatName?: string;
82 }
83
84 export interface PresenceEventPayload {
85     eventType: "USER_ONLINE" | "USER_OFFLINE" | "VOICE_JOINED" | "
        VOICE_LEFT";
86     username: string;
87     voiceChannelId?: number | null;
88 }
89
90 export interface VoiceSignalingPayload {
91     type: "OFFER" | "ANSWER" | "ICE_CANDIDATE";

```

```
92   fromUser: string;
93   toUser: string;
94   channelId: number;
95   sdp?: string;
96   candidate?: string;
97 }
```